附件 2：



华南理工大学

South China University of Technology

课程设计报告书

题目： 二号，黑体，加粗，居中

学　院 _____

专　业 _____

学生姓名 _____

学生学号 _____

指导教师 _____

课程编号 _____

课程学分 _____

起始日期 _____

封面纸推荐用 210g/m2 的绿色色书

教师评语	教师签名： 日期：
成绩评定	
备注	

局域网聊天室软件设计

选题编号 5　　Python 客户端与 C++ 服务端

一 选题背景

本课题面向同一局域网中的多人文字交流，设计并实现具有注册、登录、注销、成员列表、群发和选定多人发送能力的聊天室。系统采用客户机与服务器模式，以 UDP 组播承担群聊下行，以 TCP 承担账号控制和定向消息，并在应用协议中设置明确的任务码、错误语义和完整性校验结构。

设计目标是让通信功能、协议约束和可验证的实现相互对应。程序不仅应在正常输入下完成中文交流，还应能处理不存在的任务码、错误参数、篡改报文、TCP 拆包粘包和连接中断。交付内容包括可运行程序、完整项目源码、协议说明、真实测试记录和带讲解的运行录像。

表 1 作业要求与实现映射

要求	实现位置与行为
局域网多人通信	服务端向 239.255.42.99:45861 发送 UDP 组播，TTL 为 1
客户机与服务器模式	Python Tkinter 客户端与 C++17 Winsock 服务端
中英文文本	UTF-8 JSON，支持中文账号、中文英文正文和换行
账号与界面	注册、登录、注销、在线名单、会话记录和输入框
群发和指定多人	群发按钮；名单多选后 TCP 定向，仅目标与发送者接收
无效任务与完整性	INVALID_TASK 等错误响应；SHA-256 校验失败则不执行业务
报告与录像	详细语法语义设计、源码附录、测试证据和 720p MP4

二 方案论证

2 1 通信方式与语言分工

若全部采用 TCP 单播，群发时服务端需要逐连接重复传输正文；若全部采用 UDP，账号认证、定向隔离和可靠交付均需另外实现。结合课程对组播或广播的要求，本方案采用 TCP 控制通道与 UDP 组播群聊的组合。IPv4 组播把一个数据报发送给加入同一组的接收者，天然适合多人群发，但并不保证交付和到达顺序[1]。

Python 负责窗口、用户操作、协议封装和收包事件分发，聊天程序运行只依赖标准库。C++17 负责连接接入、账号校验、成员状态、组播发送和补发队列，使用 Windows Winsock 与 BCrypt。两端通过相同的 UTF-8 报文结构和摘要规则互通，避免依赖同一种语言的对象布局。

2 2 可靠性与安全边界

服务端对群聊保存每位在线成员的待确认项。客户端成功验包后回送 ACK；约 800 毫秒仍未确认时，服务端通过原 TCP 会话补发一次。客户端记录最近 2048 个消息标识，UDP 副本和 TCP 副本只展示一次。补发依赖仍然在线的 TCP 连接，不提供离线队列或跨重启聊天历史。

账号文件不存明文密码，而是保存 16 字节随机盐和 200000 次迭代的 PBKDF2-HMAC-SHA256 派生摘要，派生长度为 32 字节[2]。报文使用 SHA-256 检出内容改变。未采用 TLS 或带密钥的消息认证码，因此不能阻止攻击者修改内容后重新计算摘要；密码在传输链路中仍为明文。本设计的安全结论仅限课程局域网演示环境。

2 3 运行环境与部署约束

实测环境为 Windows、Python 3.14.7、MinGW-w64 GCC 16.1.0。客户端要求 Python 3.10 以上并安装 Tcl/Tk；已提供静态链接的 C++ 服务端可执行文件。默认 TCP 端口为 45860，UDP 端口为 45861，每连接最多受理 120 请求每秒，最多保持 64 条 TCP 连接，每条正文最多 2000 个 Unicode 码点。该连接上限属于实现约束，不代表已完成 64 人并发性能验证。

两台电脑联机时均接入同一局域网。客户端服务器地址填写服务端电脑的局域网 IPv4，多网卡机器指定实际组播网卡。系统需允许对应 TCP 和 UDP 端口通信。校园网 AP 隔离、交换机策略或跨网段部署可能阻止组播；此时群聊仍可经 TCP 补发到达，但该现象不能当作跨机组播验收通过。

三 过程论述

3 1 模块结构与数据流

为说明账号控制、定向消息和群聊共享同一会话状态但采用不同传输路径，图 1 给出模块及数据流。客户端主线程负责界面，网络线程通过事件队列向主线程传递收包结果，符合 Tkinter 的事件与线程使用方式[3]。

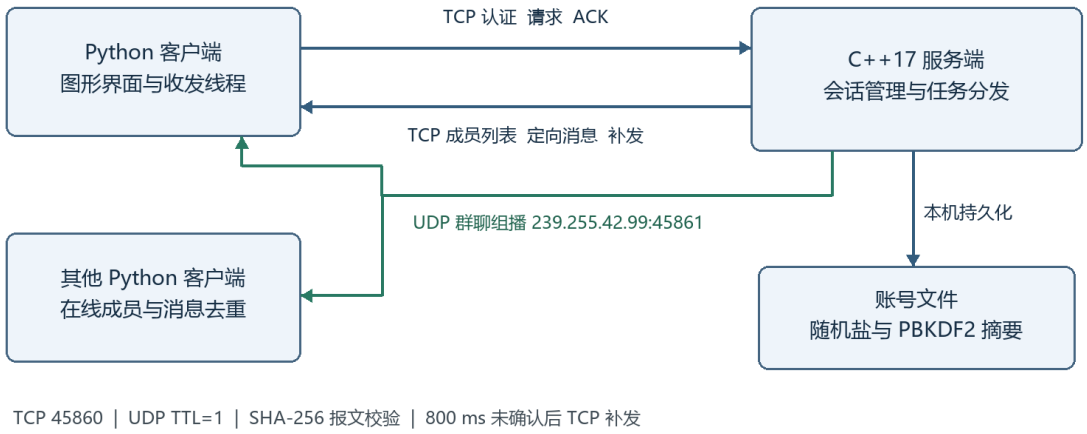


图 1 聊天室模块结构与传输路径

图中客户端向服务端发送注册、登录和聊天请求，服务端取得连接绑定的真实用户名，完成参数校验后分发消息。群聊下行进入组播组，定向消息只写入目标成员及发送者的 TCP 连接；账号存储仅在服务端本机访问。客户端每 4 秒发送心跳，服务端读超时为 15 秒、写超时为 5 秒。

表 2 工程模块与职责

文件	职责
server_cpp/main.cpp	C++ 帧解析、任务分发、账号持久化、成员管理、组播与补发
lan_chat/protocol.py	跨语言统一的报文格式、序列化、摘要、TCP 帧读取
lan_chat/client.py	TCP 连接、UDP 入组、ACK、去重与心跳
lan_chat/gui.py	账号界面、聊天输入、多人选择、状态和日志显示
lan_chat/native_server.py	启动和监控 C++ 服务进程，管理退出与日志
tests/test_chat.py	启动真实 C++ 进程并验证 TCP 和 UDP 行为

3 2 账号与会话状态

未登录连接可以注册、登录或发送心跳；注册成功不会自动登录。登录成功后，用户名与当前连接绑定，并加入在线表。同一用户名不能同时登录两条连接。注销只移除在线会话及其待确认消息，不删除账号。读到 EOF、网络错误或读超时后，服务端清理成员并推送新名单。

注册时先校验用户名和密码范围，再生成随机盐并派生摘要，将新账号写入临时文件，刷新后原子替换账号文件。账号访问加锁，避免并发注册覆盖；每个连接写入加锁，避免多个线程交错写入导致帧损坏。成员名单推送串行化，防止旧快照在新快照之后到达。

3 3 应用协议语法

协议名为 LCP 1。TCP 帧由 4 字节大端无符号长度 N 和 N 字节 UTF-8 JSON 组成，N 的有效范围为 1 至 32768。接收函数先循环读满 4 字节，再循环读满正文，因此可以处理拆包和粘包。UDP 一个数据报恰好携带一份 JSON，不含 TCP 长度前缀。

表 3 报文顶层字段

字段	语法约束	语义
v	整数 1	协议版本，不接受布尔值
id	32 位小写十六进制字符串	独立随机报文标识
type	1 至 40 字节字符串	区分大小写的任务码
ts	0 至 2^53-1 的整数	Unix 毫秒时间戳，用于展示
body	JSON 对象	具体任务参数
checksum	64 位小写十六进制字符串	其余五字段的 SHA-256 摘要

顶层只允许表 3 列出的六个字段，缺失、额外字段、重复键、非法 UTF-8 和不支持的版本均拒绝。JSON 键递归排序，使用紧凑分隔符，不转义中文；业务字段不使用浮点数。C++ 解析器还限制嵌套深度，避免过深对象占用调用栈。

3 4 客户端任务语义

表 4 客户端请求与状态约束

任务	body 主要字段	语义与前置状态
REGISTER	name, password	未登录；用户名 1 至 20 个汉字或 ASCII 字母数字及 _ -；密码 6 至 128 字符
LOGIN	name, password	未登录；校验密码并检查账号未在线；成功返回组播配置

任务	body 主要字段	语义与前置状态
LOGOUT	空对象	已登录；退出会话，保留注册账号
MEMBERS	空对象	已登录；触发完整成员名单推送
CHAT	text, mode=all, targets=[]	已登录；群发给当前所有在线成员，包含发送者
CHAT	text, mode=selected, targets	已登录；目标 1 至 64 人且全部在线；TCP 定向，回显给发送者
ACK	message_id	已登录；清除该用户的一条群聊待确认项
PING	空对象	任意连接状态；返回 PONG，保持连接活跃

CHAT 正文要求 1 至 2000 个 Unicode 码点且不能全为空白。定向目标去重，任一目标离线则整条拒绝，避免部分成员收到而发送方误判。发送者以已登录连接为准，不能由请求体伪造。每连接保留最近 512 个合法任务请求 id，重复时返回 REPLAY；该策略是有界的会话内去重，不属于跨连接防重放。

3 5 服务端响应与错误处理

表 5 服务端响应字段与业务含义

响应	主要字段与含义
RESULT	request 关联原请求，action 标识任务，ok 表示成功或失败，message 为提示
登录 RESULT	附加 name、group、udp_port、room_id，用于加入服务实例对应的聊天室
聊天 RESULT	附加 message_id，只表示服务端受理，不等于所有成员已展示
ROSTER	members 为排序后的完整名单，客户端替换旧名单
MESSAGE	room_id、sender、text、mode、targets、recipients、request；外层 id 用于 ACK 和去重
ERROR	code、message、request；无法解析原 id 时 request 为空串
PONG	request 关联心跳请求

表 6 典型错误语义与恢复方式

代码	条件	处理
INVALID_TASK	不存在的任务码	返回错误，保持连接
BAD_PACKET	JSON、UTF-8、字段、版本错误	丢弃当前已知边界的帧，继续读取
BAD_CHECKSUM	摘要不匹配	不执行业务，连接保留
FRAME_TOO_LARGE	长度 0 或超过 32768	返回错误并关闭该输入连接
BAD_BODY	类型或字符数不合法	拒绝当前任务
AUTH_FAILED	用户名不存在或密码错误	保持未登录

代码	条件	处理
ALREADY_ONLINE	账号已在另一连接登录	拒绝重复登录
NOT_LOGGED_IN	未登录执行聊天等任务	拒绝执行业务
STATE	在线状态再次注册或登录	要求先注销
NO_RECIPIENT	目标为空或包含离线账号	整条定向消息不发送
REPLAY	本连接近期请求 id 重复	不重复执行业务
RATE_LIMIT	每秒超过 120 请求	拒绝当前请求

3 6 完整性校验结构与算法

发送方构造 v、id、type、ts、body 五字段对象，递归按键名排序，以无多余空白的 UTF-8 JSON 编码得到字节串 M，计算 SHA-256(M)，将结果转为 64 位小写十六进制字符串并填入 checksum。接收方重新执行相同计算，用等长逐字节方式比较摘要。

处理顺序为：帧长度检查、UTF-8 与 JSON 解析、字段约束检查、摘要验证、任务码检查、会话与参数检查、执行业务。修改 body.text 而保留旧摘要，会返回 BAD_CHECKSUM；构造校验正确但 type 不存在的报文，会返回 INVALID_TASK。两类错误分别覆盖语法完整性和无效语义，均不使服务器退出。

3 7 组播确认与补发流程

群发受理时，服务端取得在线成员快照，为每位成员创建以用户名和 MESSAGE.id 为键的待确认记录，截止时间为当前单调时钟加 800 毫秒，然后发送一次 UDP 组播。客户端校验房间实例和实际收件名单后回 ACK，再通过最近消息 id 集合去重展示。

后台补发线程每 100 毫秒检查一次截止时间；超过截止时间仍未确认且该成员在线时，经其 TCP 连接发送原报文并删除记录。由于扫描间隔和调度影响，约 0.8 至 0.9 秒后触发补发，不构成实时性能保证。组播 ACK 丢失时可能出现重复副本，客户端只展示一次。超出链路 MTU 的 UDP 报文可能被 IP 分片，任何分片丢失均可能使该消息走 TCP 补发。

3 8 图形界面与实际运行

界面将账号操作、会话、成员和消息输入放在同一页。图 2 展示已登录状态及中文消息，图 3 用独立客户端实际收包记录验证两人定向发送和旁观者隔离。界面网络操作在后台线程执行，主线程以定时轮询事件队列更新控件；连接重试使用代次标识忽略旧连接遗留事件。

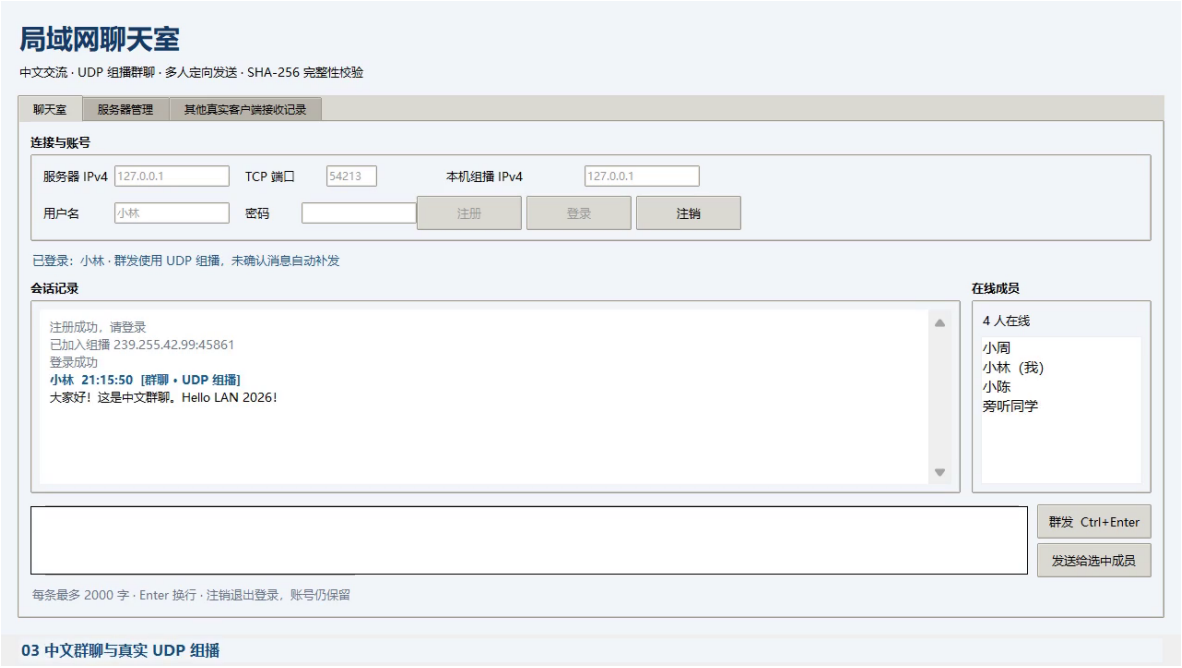


图 2 中文群聊与在线成员界面



图 3 两位目标与旁观者的实际接收记录

图 2 的传输标记用于区分 UDP 组播和 TCP 补发，不能仅凭消息到达就断定组播成功。图 3 中小周与小陈收到定向正文，旁听同学只保留群聊记录。服务器管理页显示实际 C++ 子进程输出，便于核对组播、定向、错误拒绝和补发行为。

四 结果分析

4 1 测试方法与实测结果

自动测试真实启动编译后的 C++ 服务进程，再通过 Python 客户端建立 TCP 和 UDP 连接；账号文件使用临时目录，测试结束停止进程并清理。当前 12 项测试全部通过，原始输出保存在 docs/测试结果.txt。该结果来自本机真实网络栈、多线程和多进程运行，没有把模拟收包当作实测。

表 7 自动测试覆盖与结果

编号	测试内容	结果
01	中英文、表情和换行报文编解码	通过
02	正文被修改而摘要不变，检出篡改	通过
03	JSON、UTF-8、重复键、非有限数与超长报文拒绝	通过
04	注册登录、错误密码、重复账号、重复在线及派生摘要互验	通过
05	两客户端真实 UDP 中文组播与相同 id 去重	通过
06	选中两人接收，第四个账号不接收定向消息	通过
07	关闭一客户端组播后，约 800 ms 经 TCP 补发	通过
08	不存在任务码及错误摘要后继续正常响应心跳	通过
09	两帧粘连、每次 7 字节拆分发送仍能准确解帧	通过
10	恶意格式、匿名聊天、超大帧；新连接仍正常	通过
11	空白与超长正文、目标错误、重复请求、离线名单	通过
12	服务端重启后仍能读取已注册账号登录	通过

4 2 演示结果与工程解释

演示录像使用一个 C++ 服务端、主窗口客户端和三个独立网络客户端。群聊被全部在线成员接收；定向正文只在小周与小陈的接收列出现，旁听同学未收到。将旁听同学退出组播组后仍保留 TCP 会话，再次群发时该客户端通过 TCP 收到补发。录像记录为真实 GUI 画面，中文讲解为系统语音合成。

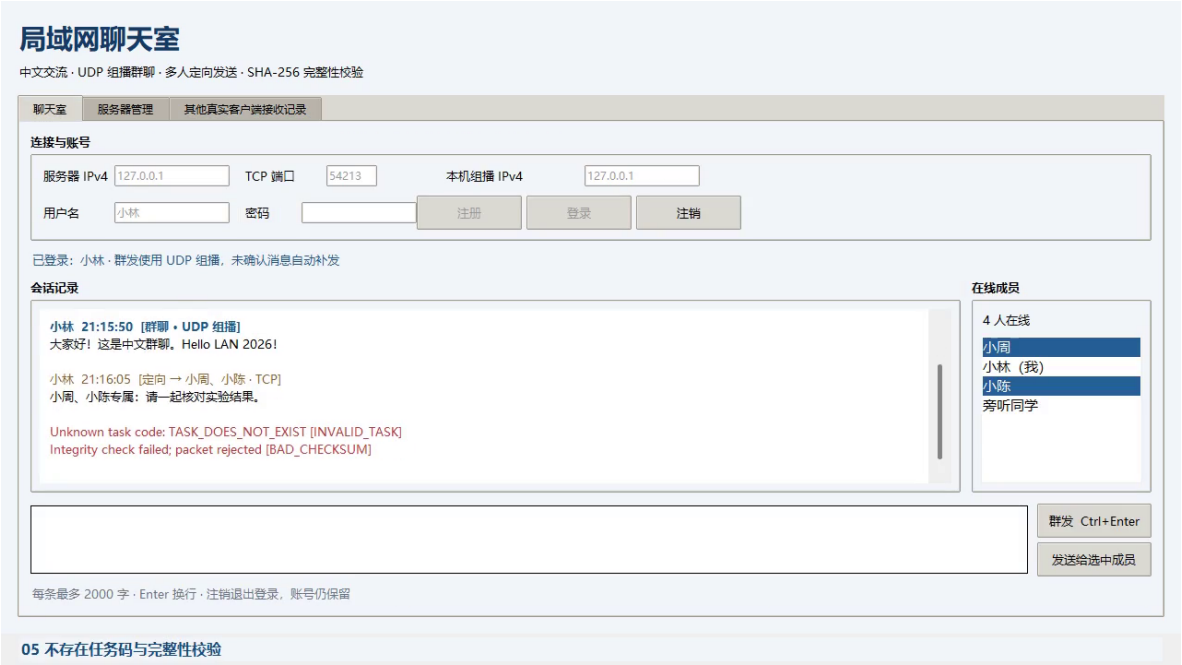


图 4 无效任务与完整性错误被拒绝后继续通信

组播降低了正常群发下行重复发送正文的次数，但仍需每位成员回 ACK，服务端也维护按成员数量增长的待确认状态。因此本实现是面向小规模课程聊天室的方案，没有据此宣称具备互联网规模吞吐能力。多线程和连接写锁保障帧不交错，不能保证所有客户端按照同一全局顺序看到并发群聊。

4 3 未验证范围与现场验收

本次未在两台实体电脑之间完成验收，也未开展 64 人负载、校园 AP 丢包率、实际链路吞吐和长期运行测试。现场应在两台同网段电脑分别启动客户端，确认成员名单、双向中文群聊、两人定向隔离和注销同步，并在日志中看到 UDP 组播标记。如果仅出现 TCP 标记，只能证明可靠回退生效，不能代替跨机组播验证。

五 课程设计总结

本设计把面向连接的控制通道与无连接的组播通道组合，实现了账号会话、多人群聊和定向发送。协议实现从字节边界、语法、完整性和语义四个层次逐步校验，使异常任务与受损报文能被明确拒绝。跨语言互通的关键在于约定规范化 JSON、整型毫秒时间戳、UTF-8 编码和统一摘要范围，而不能依赖语言默认对象表示。

实现过程中需要关注 Windows 多网卡组播选择、TCP 分段、异步界面更新和资源退出。自动测试和录像提供了可重复的正常与异常证据；后续可增加 TLS、消息认证、持久化历史、重新连接、服务器发现以及跨机器网络测试。引入这些能力时应同步更新协议版本与测试，避免仅增加界面按钮而缺少一致的服务端语义。

参考文献

[1] Deering S. Host Extensions for IP Multicasting. RFC 1112, 1989, 第 2 至 3 节. https://www.rfc-editor.org/rfc/rfc1112.txt. 访问日期 2026-10-31.

[2] Moriarty K, Kaliski B, Rusch A. PKCS #5 Password-Based Cryptography Specification Version 2.1. RFC 8018, 2017, 第 5.2 节. https://www.rfc-editor.org/rfc/rfc8018.txt. 访问日期 2026-10-31.

[3] Python Software Foundation. tkinter Python interface to Tcl/Tk, Threading model. https://docs.python.org/3/library/tkinter.html. 访问日期 2026-10-31.

[4] Lohmann N. JSON for Modern C++, version 3.11.3. https://github.com/nlohmann/json/tree/v3.11.3. MIT License. 访问日期 2026-10-31.

附录 A 核心程序完整源码

以下按文件列出本课题核心自编源码，行号对应交付工程。过长行在排版时折行，续行以 > 标识；可执行源码以工程文件为准。第三方 nlohmann/json 3.11.3 头文件体量较大，随 server_cpp/vendor/json.hpp 与 MIT 许可证独立提交，不重复印入本附录。自动测试完整源码另见附录 B。

server_cpp/main.cpp

```
// LCP/1 native Windows server. Build with MinGW-w64 or MSVC, C++17.
#define WIN32_LEAN_AND_MEAN
#define NOMINMAX
#include <winsock2.h>
#include <ws2tcpip.h>
#include <windows.h>
#include <bcrypt.h>
#include "vendor/json.hpp"
#include <algorithm>
#include <atomic>
#include <chrono>
#include <deque>
#include <filesystem>
#include <fstream>
#include <iostream>
#include <map>
#include <memory>
#include <mutex>
#include <set>
#include <string>
#include <thread>
#include <vector>

using json = nlohmann::json;
using Clock = std::chrono::steady_clock;
constexpr size_t MAX_FRAME = 32768;
struct Error : std::runtime_error {
    std::string code;
    Error(std::string c, std::string m) : runtime_error(m), code(std::move(c)) {}
};
std::mutex log_mutex;
void log(const std::string& text) {
    std::lock_guard<std::mutex> guard(log_mutex);
    std::cout << text << std::endl;
}
std::string hex(const std::vector<unsigned char>& bytes) {
    const char* digits = "0123456789abcdef";
    std::string result;
    for (auto b : bytes) { result += digits[b >> 4]; result += digits[b & 15]; }
    return result;
}
std::vector<unsigned char> random_bytes(size_t size) {
    std::vector<unsigned char> result(size);
    if (BCryptGenRandom(nullptr, result.data(), ULONG(size), BCRYPT_USE_SYSTEM_PREFERRED_RNG) < 0)
        throw std::runtime_error("BCryptGenRandom failed");
    return result;
}
std::string sha256(const std::string& input) {
    BCRYPT_ALG_HANDLE alg = nullptr;
    BCRYPT_HASH_HANDLE hash = nullptr;
    if (BCryptOpenAlgorithmProvider(&alg, BCRYPT_SHA256_ALGORITHM, nullptr, 0) < 0)
```

```
        throw std::runtime_error("SHA256 provider failed");
    std::vector<unsigned char> output(32);
    NTSTATUS status = BCryptCreateHash(alg, &hash, nullptr, 0, nullptr, 0, 0);
    if (status >= 0) status = BCryptHashData(hash, (PUCHAR)input.data(), ULONG(input.size()), 0);
    if (status >= 0) status = BCryptFinishHash(hash, output.data(), ULONG(output.size()), 0);
    if (hash) BCryptDestroyHash(hash);
    BCryptCloseAlgorithmProvider(alg, 0);
    if (status < 0) throw std::runtime_error("SHA256 failed");
    return hex(output);
}
bool equal_digest(const std::string& a, const std::string& b) {
    if (a.size() != b.size()) return false;
    unsigned char diff = 0;
    for (size_t i = 0; i < a.size(); ++i) diff |= a[i] ^ b[i];
    return diff == 0;
}
std::string derive(const std::string& password, const std::string& salt_hex) {
    if (salt_hex.size() != 32) throw std::runtime_error("invalid account salt");
    std::vector<unsigned char> salt(16), digest(32);
    for (size_t i = 0; i < 16; ++i) salt[i] = (unsigned char)std::stoul(salt_hex.substr(i * 2, 2),
nullptr, 16);
    BCRYPT_ALG_HANDLE alg = nullptr;
    if (BCryptOpenAlgorithmProvider(&alg, BCRYPT_SHA256_ALGORITHM, nullptr, BCRYPT_ALG_HANDLE_HMAC_
FLAG) < 0)
        throw std::runtime_error("PBKDF2 provider failed");
    NTSTATUS result = BCryptDeriveKeyPBKDF2(alg, (PUCHAR)password.data(), ULONG(password.size()),
                          salt.data(), ULONG(salt.size()), 200000, digest.data(), ULONG(digest.siz
e()), 0);
    BCryptCloseAlgorithmProvider(alg, 0);
    if (result < 0) throw std::runtime_error("PBKDF2 failed");
    return hex(digest);
}
// Valid JSON UTF-8 is checked by nlohmann; count Unicode code points.
std::vector<unsigned> codepoints(const std::string& s) {
    std::vector<unsigned> result;
    for (size_t i = 0; i < s.size();) {
        unsigned char c = s[i++]; unsigned cp = c; int count = 0;
        if (c >= 0xf0) { cp = c & 7; count = 3; }
        else if (c >= 0xe0) { cp = c & 15; count = 2; }
        else if (c >= 0xc0) { cp = c & 31; count = 1; }
        while (count-- && i < s.size()) cp = (cp << 6) | (s[i++] & 63);
        result.push_back(cp);
    }
    return result;
}
bool valid_name(const std::string& name) {
    auto chars = codepoints(name);
    if (chars.empty() || chars.size() > 20) return false;
    for (auto c : chars)
        if (!((c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z') || (c >= '0' && c <= '9') ||
              c == '_' || c == '-' || (c >= 0x4e00 && c <= 0x9fff))) return false;
    return true;
}
bool hexadecimal(const std::string& s, size_t size) {
    return s.size() == size && s.find_first_not_of("0123456789abcdef") == std::string::npos;
}
json packet(const std::string& type, json body = json::object()) {
    auto ms = std::chrono::duration_cast<std::chrono::milliseconds>(std::chrono::system_clock::now(
).time_since_epoch()).count();
    json p = {{"v", 1}, {"id", hex(random_bytes(16))}, {"type", type}, {"ts", ms}, {"body", body}};
    p["checksum"] = sha256(p.dump());
    return p;
}
json parse(const std::string& raw) {
    try {
        std::map<int, std::set<std::string>> keys;
```

```
        auto callback = [&keys](int depth, json::parse_event_t event, json& parsed) {
            if (depth > 32) throw Error("BAD_PACKET", "JSON nesting exceeds 32");
            if (event == json::parse_event_t::object_start) keys[depth + 1].clear();
            if (event == json::parse_event_t::key && !keys[depth].insert(parsed.get<std::string>())
 > .second)
                throw Error("BAD_PACKET", "Duplicate JSON key");
            return true;
        };
        json p = json::parse(raw, callback);
        if (!p.is_object() || p.size() != 6 || !p.contains("v") || !p.contains("id") ||
                !p.contains("type") || !p.contains("ts") || !p.contains("body") || !p.contains("che
 > cksum"))
            throw Error("BAD_PACKET", "Invalid envelope fields");
        if (!p["v"].is_number_integer() || p["v"] != 1 || !p["body"].is_object() ||
                !p["id"].is_string() || !hexadecimal(p["id"], 32) || !p["type"].is_string() ||
                p["type"].get<std::string>().empty() || p["type"].get<std::string>().size() > 40 ||
                !p["ts"].is_number_integer() || p["ts"].get<int64_t>() < 0 ||
                p["ts"].get<uint64_t>() > 9007199254740991ULL ||
                !p["checksum"].is_string() || !hexadecimal(p["checksum"], 64))
            throw Error("BAD_PACKET", "Invalid field types or values");
        std::string digest = p["checksum"];
        json unsigned_p = p; unsigned_p.erase("checksum");
        if (!equal_digest(digest, sha256(unsigned_p.dump())))
            throw Error("BAD_CHECKSUM", "Integrity check failed; packet rejected");
        return p;
    } catch (const Error&) { throw; }
    catch (const std::exception&) { throw Error("BAD_PACKET", "Malformed JSON or UTF-8"); }
}
bool read_exact(SOCKET sock, char* data, int size) {
    int offset = 0;
    while (offset < size) {
        int n = recv(sock, data + offset, size - offset, 0);
        if (n <= 0) return false;
        offset += n;
    }
    return true;
}
struct Peer {
    SOCKET sock;
    std::string name;  // Guarded by server state mutex.
    std::mutex write_mutex;
    std::atomic<bool> closed{false};
    std::deque<std::string> seen;
    std::deque<Clock::time_point> recent;
    explicit Peer(SOCKET s) : sock(s) {}
    ~Peer() { closesocket(sock); }
    bool send(const json& p) {
        std::string data = p.dump();
        if (data.size() > MAX_FRAME) throw Error("FRAME_TOO_LARGE", "Response exceeds maximum lengt
 > h");
        uint32_t size = htonl(uint32_t(data.size()));
        std::string frame((char*)&size, 4); frame += data;
        std::lock_guard<std::mutex> guard(write_mutex);
        if (closed) return false;
        size_t offset = 0;
        while (offset < frame.size()) {
            int n = ::send(sock, frame.data() + offset, int(frame.size() - offset), 0);
            if (n <= 0) return false;
            offset += n;
        }
        return true;
    }
};

class Server {
    std::mutex state, account_mutex, roster_mutex;
```

```
177     std::map<std::string, std::shared_ptr<Peer>> users;
178     std::set<std::shared_ptr<Peer>> peers;
179     struct Pending { Clock::time_point deadline; json message; };
180     std::map<std::pair<std::string, std::string>, Pending> pending;
181     std::string room_id = hex(random_bytes(16));
182     std::string db_path;
183     json accounts = json::object();
184     SOCKET listener = INVALID_SOCKET, udp = INVALID_SOCKET;
185     sockaddr_in multicast{};
186 public:
187     explicit Server(std::string path) : db_path(std::move(path)) {
188         if (std::filesystem::exists(std::filesystem::u8path(db_path))) {
189             std::ifstream in(std::filesystem::u8path(db_path));
190             in >> accounts;
191             if (!accounts.is_object()) throw std::runtime_error("Invalid account database");
192         }
193     }
194     void run(const std::string& host, int port, const std::string& iface) {
195         listener = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
196         sockaddr_in addr{}; addr.sin_family = AF_INET; addr.sin_port = htons((u_short)port);
197         if (inet_pton(AF_INET, host.c_str(), &addr.sin_addr) != 1)
198             throw std::runtime_error("Invalid listening IPv4");
199         if (bind(listener, (sockaddr*)&addr, sizeof(addr)) != 0 || listen(listener, 32) != 0)
200             throw std::runtime_error("Cannot bind/listen TCP port (already in use?)");
201         udp = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
202         DWORD ttl = 1; BOOL loop = TRUE;
203         setsockopt(udp, IPPROTO_IP, IP_MULTICAST_TTL, (char*)&ttl, sizeof(ttl));
204         setsockopt(udp, IPPROTO_IP, IP_MULTICAST_LOOP, (char*)&loop, sizeof(loop));
205         in_addr outgoing{};
206         if (inet_pton(AF_INET, iface.c_str(), &outgoing) != 1)
207             throw std::runtime_error("Invalid multicast interface IPv4");
208         if (setsockopt(udp, IPPROTO_IP, IP_MULTICAST_IF, (char*)&outgoing, sizeof(outgoing)) != 0)
209             throw std::runtime_error("Cannot select multicast interface");
210         multicast.sin_family = AF_INET; multicast.sin_port = htons(45861);
211         inet_pton(AF_INET, "239.255.42.99", &multicast.sin_addr);
212         std::thread([this] { repair(); }).detach();
213         log("READY TCP " + host + ":" + std::to_string(port) + " | UDP 239.255.42.99:45861 | C++17"
  > );
214         for (;;) {
215             SOCKET sock = accept(listener, nullptr, nullptr);
216             if (sock == INVALID_SOCKET) continue;
217             auto peer = std::make_shared<Peer>(sock);
218             DWORD timeout = 15000, send_timeout = 5000;
219             setsockopt(sock, SOL_SOCKET, SO_RCVTIMEO, (char*)&timeout, sizeof(timeout));
220             setsockopt(sock, SOL_SOCKET, SO_SNDTIMEO, (char*)&send_timeout, sizeof(send_timeout));
221             {
222                 std::lock_guard<std::mutex> guard(state);
223                 if (peers.size() >= 64) continue;
224                 peers.insert(peer);
225             }
226             std::thread([this, peer] { serve(peer); }).detach();
227         }
228     }
229 private:
230     void error(const std::shared_ptr<Peer>& p, const Error& e, const std::string& rid = "") {
231         p->send(packet("ERROR", {{"code", e.code}, {"message", e.what()}, {"request", rid}}));
232     }
233     void serve(const std::shared_ptr<Peer>& peer) {
234         try {
235             while (!peer->closed) {
236                 uint32_t size;
237                 if (!read_exact(peer->sock, (char*)&size, 4)) break;
238                 size = ntohl(size);
239                 if (!size || size > MAX_FRAME) {
240                     error(peer, Error("FRAME_TOO_LARGE", "TCP frame length must be 1..32768"));
241                     break;
```

```
                }
                std::string raw(size, '\0');
                if (!read_exact(peer->sock, raw.data(), int(size))) break;
                std::string rid;
                try {
                    json p = parse(raw); rid = p["id"];
                    handle(peer, p);
                } catch (const Error& e) {
                    error(peer, e, rid);
                    log("REJECT " + e.code);
                } catch (const json::exception&) {
                    error(peer, Error("BAD_BODY", "Task body has invalid field types"), rid);
                }
            }
        } catch (const std::exception& e) { log(std::string("CONNECTION ERROR ") + e.what()); }
        drop(peer);
    }
    void save_accounts() {
        auto path = std::filesystem::u8path(db_path);
        if (path.has_parent_path()) std::filesystem::create_directories(path.parent_path());
        auto temporary = path; temporary += ".tmp";
        {
            std::ofstream out(temporary, std::ios::binary | std::ios::trunc);
            out << accounts.dump(2); out.flush();
            if (!out) throw std::runtime_error("Cannot write account database");
        }
        if (!MoveFileExW(temporary.c_str(), path.c_str(), MOVEFILE_REPLACE_EXISTING | MOVEFILE_WRIT
 > E_THROUGH))
            throw std::runtime_error("Cannot commit account database");
    }
    void handle(const std::shared_ptr<Peer>& peer, const json& p) {
        std::string task = p["type"], rid = p["id"];
        json body = p["body"];
        static const std::set<std::string> tasks{"REGISTER", "LOGIN", "LOGOUT", "MEMBERS", "CHAT",
 > "ACK", "PING"};
        if (!tasks.count(task)) throw Error("INVALID_TASK", "Unknown task code: " + task);
        if (std::find(peer->seen.begin(), peer->seen.end(), rid) != peer->seen.end())
            throw Error("REPLAY", "Duplicate request identifier");
        peer->seen.push_back(rid); if (peer->seen.size() > 512) peer->seen.pop_front();
        auto now = Clock::now();
        while (!peer->recent.empty() && now - peer->recent.front() > std::chrono::seconds(1)) peer-
 >recent.pop_front();
        if (peer->recent.size() >= 120) throw Error("RATE_LIMIT", "Too many requests");
        peer->recent.push_back(now);
        std::string name;
        { std::lock_guard<std::mutex> guard(state); name = peer->name; }
        if (task == "REGISTER" || task == "LOGIN") {
            if (!name.empty()) throw Error("STATE", "Logout before switching accounts");
            if (!body.contains("name") || !body["name"].is_string() || !body.contains("password") |
 > | !body["password"].is_string())
                throw Error("BAD_BODY", "name and password must be strings");
            std::string requested = body["name"], password = body["password"];
            if (!valid_name(requested) || codepoints(password).size() < 6 || codepoints(password).s
 > ize() > 128)
                throw Error("BAD_BODY", "Name: 1..20 Chinese/ASCII letters/digits/_/-; password: 6.
 > .128 characters");
            if (task == "REGISTER") {
                std::lock_guard<std::mutex> guard(account_mutex);
                bool ok = !accounts.contains(requested);
                if (ok) {
                    auto salt = hex(random_bytes(16));
                    accounts[requested] = {{"salt", salt}, {"digest", derive(password, salt)}};
                    try { save_accounts(); }
                    catch (...) { accounts.erase(requested); throw; }
                }
                peer->send(packet("RESULT", {{"request", rid}, {"action", task}, {"ok", ok},
```

```
                    {"message", ok ? "注册成功，请登录" : "用户名已存在"}}));
                return;
            }
            bool ok = false;
            {
                std::lock_guard<std::mutex> guard(account_mutex);
                if (accounts.contains(requested)) {
                    const json& row = accounts[requested];
                    ok = equal_digest(derive(password, row.at("salt")), row.at("digest"));
                }
            }
            if (!ok) throw Error("AUTH_FAILED", "用户名或密码错误");
            {
                std::lock_guard<std::mutex> guard(state);
                if (users.count(requested)) throw Error("ALREADY_ONLINE", "该账号已经在线");
                peer->name = requested; users[requested] = peer;
            }
            peer->send(packet("RESULT", {{"request", rid}, {"action", task}, {"ok", true}, {"name",
 requested},
                  {"group", "239.255.42.99"}, {"udp_port", 45861}, {"room_id", room_id}, {"message"
, "登录成功"}}));
            log("LOGIN " + requested); roster(); return;
        }
        if (task == "PING") { peer->send(packet("PONG", {{"request", rid}})); return; }
        if (name.empty()) throw Error("NOT_LOGGED_IN", "请先登录");
        if (task == "LOGOUT") {
            {
                std::lock_guard<std::mutex> guard(state);
                users.erase(name); peer->name.clear();
                clear_pending(name);
            }
            peer->send(packet("RESULT", {{"request", rid}, {"action", task}, {"ok", true}, {"messag
e", "已注销登录"}}));
            log("LOGOUT " + name); roster();
        } else if (task == "MEMBERS") { roster(); }
        else if (task == "ACK") {
            if (!body.contains("message_id") || !body["message_id"].is_string()) throw Error("BAD_B
ODY", "message_id must be a string");
            std::lock_guard<std::mutex> guard(state);
            pending.erase({name, body["message_id"].get<std::string>()});
        } else if (task == "CHAT") { chat(peer, body, rid, name); }
    }
    void chat(const std::shared_ptr<Peer>& sender, const json& body, const std::string& rid, const
std::string& name) {
        if (!body.contains("text") || !body["text"].is_string() || !body.contains("mode") || !body[
"mode"].is_string())
            throw Error("BAD_BODY", "text and mode must be strings");
        std::string text = body["text"], mode = body["mode"];
        auto chars = codepoints(text);
        bool blank = chars.empty() || std::all_of(chars.begin(), chars.end(), [](unsigned c) {
            return c == 32 || (c >= 9 && c <= 13) || c == 0xa0 || c == 0x1680 ||
                   (c >= 0x2000 && c <= 0x200a) || c == 0x2028 || c == 0x2029 || c == 0x202f || c =
= 0x205f || c == 0x3000;
        });
        if (blank || chars.size() > 2000) throw Error("BAD_BODY", "消息须为 1 至 2000 个字符且不能
全为空白");
        json targets = body.value("targets", json::array());
        if ((mode != "all" && mode != "selected") || !targets.is_array() || targets.size() > 64)
            throw Error("BAD_BODY", "Invalid mode or targets");
        std::set<std::string> requested;
        for (const auto& target : targets) {
            if (!target.is_string()) throw Error("BAD_BODY", "Recipient must be a string");
            requested.insert(target.get<std::string>());
        }
        std::vector<std::shared_ptr<Peer>> recipients;
        std::vector<std::string> names;
```

```
360         json message;
361         {
362             std::lock_guard<std::mutex> guard(state);
363             if (mode == "all") {
364                 if (!targets.empty()) throw Error("BAD_BODY", "Group message targets must be empty"
 > );
365                 for (const auto& entry : users) names.push_back(entry.first);
366             } else {
367                 if (requested.empty()) throw Error("NO_RECIPIENT", "请选择在线接收人");
368                 for (const auto& target : requested) if (!users.count(target)) throw Error("NO_RECI
 > PIENT", "接收人已离线");
369                 requested.insert(name); names.assign(requested.begin(), requested.end());
370             }
371             for (const auto& target : names) recipients.push_back(users.at(target));
372             message = packet("MESSAGE", {{"room_id", room_id}, {"sender", name}, {"text", text}, {"
 > mode", mode},
373                              {"targets", targets}, {"recipients", names}, {"request", rid}});
374             if (message.dump().size() > MAX_FRAME) throw Error("FRAME_TOO_LARGE", "Encoded message
 > exceeds maximum length");
375             if (mode == "all") for (const auto& target : names)
376                 pending[{target, message["id"].get<std::string>()}] = {Clock::now() + std::chrono::
 > milliseconds(800), message};
377         }
378         if (mode == "all") {
379             std::string data = message.dump();
380             int sent = sendto(udp, data.data(), int(data.size()), 0, (sockaddr*)&multicast, sizeof(
 > multicast));
381             log("MULTICAST " + name + " -> " + std::to_string(names.size()) + " recipients, " +
382                 std::to_string(data.size()) + " bytes" + (sent < 0 ? " (failed; TCP repair pending)
 > " : ""));
383         } else {
384             for (auto& peer : recipients) if (!peer->send(message)) drop(peer);
385             log("SELECTED " + name + " -> " + targets.dump());
386         }
387         sender->send(packet("RESULT", {{"request", rid}, {"action", "CHAT"}, {"ok", true},
388                                 {"message", "服务器已受理"}, {"message_id", message["id"]}}))
 > ;
389     }
390     void clear_pending(const std::string& name) {  // Caller holds state mutex.
391         for (auto it = pending.begin(); it != pending.end();)
392             if (it->first.first == name) it = pending.erase(it); else ++it;
393     }
394     void drop(const std::shared_ptr<Peer>& peer) {
395         if (peer->closed.exchange(true)) return;
396         shutdown(peer->sock, SD_BOTH);
397         bool changed;
398         std::string name;
399         {
400             std::lock_guard<std::mutex> guard(state);
401             name = peer->name; changed = !name.empty();
402             if (changed) { users.erase(name); clear_pending(name); }
403             peer->name.clear(); peers.erase(peer);
404         }
405         if (changed) { log("DISCONNECT " + name); roster(); }
406     }
407     void roster() {
408         std::vector<std::shared_ptr<Peer>> failed;
409         {
410             std::lock_guard<std::mutex> serialize(roster_mutex);
411             std::vector<std::string> names;
412             std::vector<std::shared_ptr<Peer>> clients;
413             {
414                 std::lock_guard<std::mutex> guard(state);
415                 for (const auto& entry : users) { names.push_back(entry.first); clients.push_back(e
 > ntry.second); }
416             }
```

```
            auto message = packet("ROSTER", {{"members", names}});
            for (auto& peer : clients) if (!peer->send(message)) failed.push_back(peer);
        }
        for (auto& peer : failed) drop(peer);
    }
    void repair() {
        for (;;) {
            std::this_thread::sleep_for(std::chrono::milliseconds(100));
            std::vector<std::pair<std::shared_ptr<Peer>, json>> due;
            {
                std::lock_guard<std::mutex> guard(state);
                for (auto it = pending.begin(); it != pending.end();) {
                    if (it->second.deadline <= Clock::now()) {
                        auto user = users.find(it->first.first);
                        if (user != users.end()) due.emplace_back(user->second, it->second.message)
 > ;
                        it = pending.erase(it);
                    } else ++it;
                }
            }
            for (auto& entry : due) {
                if (!entry.first->send(entry.second)) drop(entry.first);
                log("TCP_REPAIR " + entry.second["id"].get<std::string>().substr(0, 8));
            }
        }
    }
};
int wmain(int argc, wchar_t** argv) {
    SetConsoleOutputCP(CP_UTF8);
    WSADATA wsa{};
    if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0) return 1;
    try {
        std::string host = "0.0.0.0", iface = "0.0.0.0", db = "data/accounts.json";
        int port = 45860;
        std::vector<std::string> arguments;
        for (int i = 0; i < argc; ++i) {
            int size = WideCharToMultiByte(CP_UTF8, 0, argv[i], -1, nullptr, 0, nullptr, nullptr);
            std::string value(size, '\0');
            WideCharToMultiByte(CP_UTF8, 0, argv[i], -1, value.data(), size, nullptr, nullptr);
            value.pop_back(); arguments.push_back(value);
        }
        for (int i = 1; i < argc; ++i) {
            std::string arg = arguments[i];
            if (arg == "--help") { std::cout << "server.exe [--host IPv4] [--port N] [--interface I
 > Pv4] [--db PATH]\n"; return 0; }
            if (++i >= argc) throw std::runtime_error("Missing argument value");
            if (arg == "--host") host = arguments[i];
            else if (arg == "--port") port = std::stoi(arguments[i]);
            else if (arg == "--interface") iface = arguments[i];
            else if (arg == "--db") db = arguments[i];
            else throw std::runtime_error("Unknown option: " + arg);
        }
        if (port < 1 || port > 65535) throw std::runtime_error("Port must be 1..65535");
        Server server(db); server.run(host, port, iface);
    } catch (const std::exception& e) { std::cerr << "FATAL " << e.what() << std::endl; return 1; }
    return 0;
}
```

lan_chat/protocol.py

```python
"""LCP/1 framing and integrity checks shared by TCP and UDP."""
import hashlib
import hmac
import json
import math
import re
import struct
import time
import uuid

MAX_FRAME = 32768
MAX_TEXT = 2000
GROUP = "239.255.42.99"
UDP_PORT = 45861
TCP_PORT = 45860


class ProtocolError(ValueError):
    def __init__(self, code, message):
        super().__init__(message)
        self.code = code


def canonical(value):
    return json.dumps(value, ensure_ascii=False, sort_keys=True,
                      separators=(",", ":"), allow_nan=False).encode("utf-8")


def make_packet(task, body=None):
    obj = dict(v=1, id=uuid.uuid4().hex, type=task, ts=int(time.time() * 1000), body=body or {})
    obj["checksum"] = hashlib.sha256(canonical(obj)).hexdigest()
    return obj


def encode(packet):
    data = canonical(packet)
    if len(data) > MAX_FRAME:
        raise ProtocolError("FRAME_TOO_LARGE", "报文超过 32768 字节")
    return data


def _pairs(items):
    result = {}
    for key, value in items:
        if key in result:
            raise ValueError("duplicate key")
        result[key] = value
    return result


def decode(data):
    if len(data) > MAX_FRAME:
        raise ProtocolError("FRAME_TOO_LARGE", "报文过长")
    try:
        obj = json.loads(data.decode("utf-8"), object_pairs_hook=_pairs,
                         parse_constant=lambda x: (_ for _ in ()).throw(ValueError(x)))
        if not isinstance(obj, dict) or set(obj) != {"v", "id", "type", "ts", "body", "checksum"}:
            raise ValueError("fields")
        if type(obj["v"]) is not int or obj["v"] != 1:
            raise ValueError("version")
        if not isinstance(obj["id"], str) or not re.fullmatch(r"[0-9a-f]{32}", obj["id"]):
            raise ValueError("id")
```

```
        if not isinstance(obj["type"], str) or not 1 <= len(obj["type"]) <= 40:
            raise ValueError("type")
        if type(obj["ts"]) is not int or not 0 <= obj["ts"] <= 9007199254740991:
            raise ValueError("timestamp")
        if not isinstance(obj["body"], dict):
            raise ValueError("body")
        checksum = obj["checksum"]
        if not isinstance(checksum, str) or not re.fullmatch(r"[0-9a-f]{64}", checksum):
            raise ValueError("checksum")
        unsigned = {k: v for k, v in obj.items() if k != "checksum"}
        expected = hashlib.sha256(canonical(unsigned)).hexdigest()
        if not hmac.compare_digest(checksum, expected):
            raise ProtocolError("BAD_CHECKSUM", "完整性校验失败，报文已拒绝")
        return obj
    except ProtocolError:
        raise
    except (ValueError, TypeError, UnicodeError, RecursionError, OverflowError) as exc:
        raise ProtocolError("BAD_PACKET", "报文语法或字段格式无效") from exc


def send_frame(sock, packet):
    data = encode(packet)
    sock.sendall(struct.pack("!I", len(data)) + data)


def recv_exact(sock, size):
    data = bytearray()
    while len(data) < size:
        block = sock.recv(size - len(data))
        if not block:
            raise EOFError("连接已关闭")
        data.extend(block)
    return bytes(data)


def recv_frame(sock):
    size = struct.unpack("!I", recv_exact(sock, 4))[0]
    if not 1 <= size <= MAX_FRAME:
        raise ProtocolError("FRAME_TOO_LARGE", "TCP 帧长度不合法")
    return decode(recv_exact(sock, size))
```

lan_chat/client.py

```
"""Headless client used by both Tkinter and integration tests."""
from collections import deque
import socket
import threading

from .protocol import TCP_PORT, ProtocolError, decode, make_packet, recv_frame, send_frame


class ChatClient:
    def __init__(self, callback, interface="0.0.0.0", multicast=True):
        self.callback = callback
        self.interface, self.multicast = interface, multicast
        self.sock = self.udp = None
        self.name = self.room_id = None
        self.lock = threading.Lock()
        self.seen_lock = threading.Lock()
        self.seen, self.order = set(), deque()
        self.stopped = threading.Event()
        self.udp_stopped = threading.Event()

    def connect(self, host, port=TCP_PORT):
        self.sock = socket.create_connection((host, port), timeout=5)
        self.sock.settimeout(None)
        threading.Thread(target=self._receive, daemon=True).start()
        threading.Thread(target=self._heartbeat, daemon=True).start()

    def request(self, task, body=None):
        packet = make_packet(task, body)
        with self.lock:
            if not self.sock or self.stopped.is_set():
                raise OSError("尚未连接服务器")
            send_frame(self.sock, packet)
        return packet["id"]

    def register(self, name, password):
        return self.request("REGISTER", dict(name=name, password=password))

    def login(self, name, password):
        return self.request("LOGIN", dict(name=name, password=password))

    def chat(self, text, targets=None):
        return self.request("CHAT", dict(text=text, mode="selected" if targets is not None else "al
 > l",
                                         targets=targets or []))

    def logout(self):
        return self.request("LOGOUT")

    def _heartbeat(self):
        while not self.stopped.wait(4):
            try:
                self.request("PING")
            except OSError:
                self.close()
                return

    def _receive(self):
        try:
            while not self.stopped.is_set():
                try:
                    packet = recv_frame(self.sock)
                except ProtocolError as exc:
```

```
                    self.callback("warning", dict(message=str(exc)))
                    if exc.code == "FRAME_TOO_LARGE":
                        break
                    continue
                body = packet["body"]
                if packet["type"] == "RESULT" and body.get("ok"):
                    if body.get("action") == "LOGIN":
                        self.name, self.room_id = body["name"], body["room_id"]
                        self._join(body["group"], body["udp_port"])
                    elif body.get("action") == "LOGOUT":
                        self.name = self.room_id = None
                        self._leave()
                if packet["type"] == "MESSAGE":
                    self._message(packet, "TCP")
                elif packet["type"] != "PONG":
                    self.callback(packet["type"].lower(), body)
        except (OSError, EOFError) as exc:
            if not self.stopped.is_set():
                self.callback("warning", dict(message=f"连接中断：{exc}"))
        finally:
            self.close()

    def _join(self, group, port):
        self._leave()
        if not self.multicast:
            self.callback("transport", dict(message="组播已关闭，使用 TCP 补发"))
            return
        udp = socket.socket(socket.AF_INET, socket.SOCK_DGRAM, socket.IPPROTO_UDP)
        try:
            udp.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
            udp.bind(("", port))
            udp.setsockopt(socket.IPPROTO_IP, socket.IP_ADD_MEMBERSHIP,
                           socket.inet_aton(group) + socket.inet_aton(self.interface))
            udp.settimeout(0.5)
        except OSError as exc:
            udp.close()
            self.callback("warning", dict(message=f"加入组播失败，自动使用 TCP 补发：{exc}"))
            return
        self.udp = udp
        self.udp_stopped = threading.Event()
        threading.Thread(target=self._receive_udp, args=(udp, self.udp_stopped), daemon=True).start
 > ()
        self.callback("transport", dict(message=f"已加入组播 {group}:{port}"))

    def _leave(self):
        self.udp_stopped.set()
        if self.udp:
            self.udp.close()
            self.udp = None

    def _receive_udp(self, udp, stopped):
        while not stopped.is_set() and not self.stopped.is_set():
            try:
                data, _ = udp.recvfrom(32769)
                packet = decode(data)
                if packet["type"] == "MESSAGE":
                    self._message(packet, "UDP 组播")
            except socket.timeout:
                continue
            except ProtocolError as exc:
                self.callback("warning", dict(message=f"丢弃异常组播：{exc}"))
            except OSError:
                break

    def _message(self, packet, transport):
        body, mid = packet["body"], packet["id"]
```

```
        if (body.get("room_id") != self.room_id or self.name not in body.get("recipients", [])
                or not isinstance(body.get("text"), str)):
            return
        if body.get("mode") == "all":
            try:
                self.request("ACK", dict(message_id=mid))
            except OSError:
                return
        with self.seen_lock:
            if mid in self.seen:
                return
            self.seen.add(mid)
            self.order.append(mid)
            if len(self.order) > 2048:
                self.seen.discard(self.order.popleft())
        self.callback("message", dict(body, transport=transport, id=mid, ts=packet["ts"]))

    def close(self):
        with self.lock:
            if self.stopped.is_set():
                return
            self.stopped.set()
            self._leave()
            if self.sock:
                try:
                    self.sock.shutdown(socket.SHUT_RDWR)
                except OSError:
                    pass
                self.sock.close()
        self.name = self.room_id = None
        self.callback("disconnected", {})
```

lan_chat/gui.py

```
"""Chinese Tkinter GUI. Network workers never access Tk widgets."""
import argparse
from datetime import datetime
from pathlib import Path
import queue
import threading
import tkinter as tk
from tkinter import messagebox, scrolledtext, ttk

from .client import ChatClient
from .protocol import MAX_TEXT, TCP_PORT, ProtocolError
from .native_server import NativeServer


class ChatApp:
    def __init__(self, root, data="data", title="局域网聊天室"):
        self.root, self.data = root, Path(data)
        self.root.title(title)
        self.root.geometry("1080x750")
        self.root.minsize(850, 650)
        self.events = queue.Queue()
        self.client = self.server = None
        self.busy = False
        self.generation = 0
        self.members = []
        self.host = tk.StringVar(value="127.0.0.1")
        self.port = tk.StringVar(value=str(TCP_PORT))
        self.interface = tk.StringVar(value="0.0.0.0")
        self.username = tk.StringVar()
        self.password = tk.StringVar()
        self.status = tk.StringVar(value="未登录 · 在服务器电脑启动服务后连接")
        self.server_host = tk.StringVar(value="0.0.0.0")
        self.server_port = tk.StringVar(value=str(TCP_PORT))
        self.server_interface = tk.StringVar(value="0.0.0.0")
        style = ttk.Style()
        style.theme_use("clam")
        style.configure("TFrame", background="#f3f6fa")
        style.configure("TLabel", background="#f3f6fa", font=("Microsoft YaHei UI", 10))
        style.configure("TButton", font=("Microsoft YaHei UI", 10), padding=(10, 6))
        style.configure("Header.TLabel", font=("Microsoft YaHei UI", 22, "bold"), foreground="#173c
> 66")
        style.configure("TLabelframe", background="#f3f6fa")
        style.configure("TLabelframe.Label", background="#f3f6fa", font=("Microsoft YaHei UI", 10,
> "bold"))
        self._build()
        root.protocol("WM_DELETE_WINDOW", self.close)
        root.after(60, self._poll)

    def _build(self):
        outer = ttk.Frame(self.root, padding=18)
        outer.pack(fill="both", expand=True)
        ttk.Label(outer, text="局域网聊天室", style="Header.TLabel").pack(anchor="w")
        ttk.Label(outer, text="中文交流 · UDP 组播群聊 · 多人定向发送 · SHA-256 完整性校验").pack(a
> nchor="w", pady=(4, 14))
        tabs = ttk.Notebook(outer)
        tabs.pack(fill="both", expand=True)
        chat, monitor = ttk.Frame(tabs, padding=12), ttk.Frame(tabs, padding=12)
        tabs.add(chat, text="  聊天室  ")
        tabs.add(monitor, text="  服务器管理  ")
        self.tabs = tabs
        box = ttk.LabelFrame(chat, text="连接与账号", padding=10)
        box.pack(fill="x")
```

```
        fields = [("服务器 IPv4", self.host, 17), ("TCP 端口", self.port, 7),
                  ("本机组播 IPv4", self.interface, 17)]
        self.connection_entries = []
        for col, (label, var, width) in enumerate(fields):
            ttk.Label(box, text=label).grid(row=0, column=2*col, padx=(0, 5))
            entry = ttk.Entry(box, textvariable=var, width=width)
            entry.grid(row=0, column=2*col+1, padx=(0, 15))
            self.connection_entries.append(entry)
        ttk.Label(box, text="用户名").grid(row=1, column=0, pady=(10, 0), sticky="w")
        self.name_entry = ttk.Entry(box, textvariable=self.username, width=17)
        self.name_entry.grid(row=1, column=1, pady=(10, 0), sticky="w")
        ttk.Label(box, text="密码").grid(row=1, column=2, pady=(10, 0), sticky="w")
        self.pass_entry = ttk.Entry(box, textvariable=self.password, show="●", width=17)
        self.pass_entry.grid(row=1, column=3, pady=(10, 0), sticky="w")
        actions = ttk.Frame(box)
        actions.grid(row=1, column=4, columnspan=2, pady=(10, 0), sticky="w")
        self.register_button = ttk.Button(actions, text="注册", command=lambda: self.auth("REGISTER
"))
        self.register_button.pack(side="left", padx=(0, 5))
        self.login_button = ttk.Button(actions, text="登录", command=lambda: self.auth("LOGIN"))
        self.login_button.pack(side="left", padx=(0, 5))
        self.logout_button = ttk.Button(actions, text="注销", command=self.logout, state="disabled"
)
        self.logout_button.pack(side="left")
        ttk.Label(chat, textvariable=self.status, foreground="#245780").pack(fill="x", pady=9)
        body = ttk.Frame(chat)
        history_box = ttk.LabelFrame(body, text="会话记录", padding=8)
        history_box.pack(side="left", fill="both", expand=True, padx=(0, 12))
        self.history = scrolledtext.ScrolledText(history_box, height=12, width=60, wrap="word", sta
te="disabled",
                                 font=("Microsoft YaHei UI", 10), bg="white", relief="flat", padx=10,
pady=8)
        self.history.pack(fill="both", expand=True)
        self.history.tag_configure("system", foreground="#6d7784")
        self.history.tag_configure("sender", foreground="#175c8c", font=("Microsoft YaHei UI", 10,
"bold"))
        self.history.tag_configure("private", foreground="#84602a")
        self.history.tag_configure("error", foreground="#b13e40")
        sidebar = ttk.LabelFrame(body, text="在线成员", padding=8)
        sidebar.pack(side="right", fill="y")
        self.member_label = ttk.Label(sidebar, text="0 人在线")
        self.member_label.pack(anchor="w", pady=(0, 6))
        self.member_list = tk.Listbox(sidebar, selectmode="multiple", exportselection=False,
                                      width=19, font=("Microsoft YaHei UI", 11), relief="flat",
                                      selectbackground="#265f96", activestyle="none")
        self.member_list.pack(fill="both", expand=True)
        ttk.Label(sidebar, text="单击可多选\n 选中成员后定向发送", foreground="#677485").pack(anchor
="w", pady=(8, 0))
        composer = ttk.Frame(chat)
        self.composer = tk.Text(composer, height=3, wrap="word", font=("Microsoft YaHei UI", 11),
                                relief="solid", borderwidth=1, padx=8, pady=6)
        self.composer.pack(side="left", fill="x", expand=True, padx=(0, 10))
        self.composer.bind("<Control-Return>", lambda event: self.send(False))
        buttons = ttk.Frame(composer)
        buttons.pack(side="right", fill="y")
        self.all_button = ttk.Button(buttons, text="群发  Ctrl+Enter", command=lambda: self.send(Fa
lse), state="disabled")
        self.all_button.pack(fill="x", pady=(0, 5))
        self.selected_button = ttk.Button(buttons, text="发送给选中成员", command=lambda: self.send
(True), state="disabled")
        self.selected_button.pack(fill="x")
        ttk.Label(chat, text="每条最多 2000 字 · Enter 换行 · 注销退出登录，账号仍保留", foreground
="#677485").pack(side="bottom", anchor="w", pady=(7, 0))
        composer.pack(side="bottom", fill="x", pady=(12, 0))
        body.pack(fill="both", expand=True)
        config = ttk.LabelFrame(monitor, text="服务配置", padding=12)
```

```
        config.pack(fill="x")
        for row, (label, var) in enumerate([("监听 IPv4", self.server_host), ("TCP 端口", self.serv
> er_port),
                                            ("组播出口 IPv4", self.server_interface)]):
            ttk.Label(config, text=label).grid(row=row, column=0, sticky="w", pady=5)
            ttk.Entry(config, textvariable=var, width=24).grid(row=row, column=1, padx=10, pady=5)
        self.start_button = ttk.Button(config, text="启动服务器", command=self.start_server)
        self.start_button.grid(row=0, column=2, padx=14)
        self.stop_button = ttk.Button(config, text="停止服务器", command=self.stop_server, state="d
> isabled")
        self.stop_button.grid(row=1, column=2, padx=14)
        ttk.Label(monitor, text="同一局域网运行一个服务器。其他电脑填写服务器的局域网 IPv4；多网卡
> 时填写实际网卡地址。",
                  wraplength=840).pack(anchor="w", pady=12)
        self.server_log = scrolledtext.ScrolledText(monitor, state="disabled", wrap="word",
                    font=("Consolas", 11), bg="#16283d", fg="#d9e9f8", padx=12, pady=12)
        self.server_log.pack(fill="both", expand=True)

    def append(self, text, tag="system"):
        self.history.configure(state="normal")
        self.history.insert("end", text + "\n", tag)
        self.history.see("end")
        self.history.configure(state="disabled")

    def _online(self, online):
        for widget in (self.name_entry, self.pass_entry, self.register_button, self.login_button, *
> self.connection_entries):
            widget.configure(state="disabled" if online else "normal")
        for widget in (self.logout_button, self.all_button, self.selected_button):
            widget.configure(state="normal" if online else "disabled")

    def _validate_connection(self):
        port = int(self.port.get())
        if not 1 <= port <= 65535:
            raise ValueError("端口须在 1 至 65535 之间")
        return self.host.get().strip(), port, self.interface.get().strip()

    def auth(self, action):
        if self.busy:
            return
        name, password = self.username.get().strip(), self.password.get()
        if not name or not password:
            self.append("请填写用户名和密码", "error")
            return
        try:
            host, port, interface = self._validate_connection()
        except ValueError as exc:
            self.append(str(exc), "error")
            return
        # One connection per auth attempt avoids stale server/interface settings.
        if self.client:
            self.generation += 1
            self.client.close()
        self.generation += 1
        generation = self.generation
        self.busy = True
        self.status.set("正在连接服务器…")
        self.client = ChatClient(lambda event, body: self.events.put((generation, event, body)), in
> terface)
        client = self.client

        def work():
            try:
                client.connect(host, port)
                client.request(action, dict(name=name, password=password))
            except (OSError, ValueError) as exc:
```

```
                self.events.put((generation, "warning", dict(message=str(exc))))
                client.close()
        threading.Thread(target=work, daemon=True).start()

    def logout(self):
        try:
            self.client.logout()
        except (OSError, AttributeError) as exc:
            self.append(str(exc), "error")

    def send(self, selected):
        text = self.composer.get("1.0", "end-1c")
        targets = [self.members[i] for i in self.member_list.curselection()] if selected else None
        if selected and not targets:
            self.append("请先在成员列表中选择一个或多个接收人", "error")
            return "break"
        if not text.strip() or len(text) > MAX_TEXT:
            self.append("消息须为 1 至 2000 个字符且不能全为空白", "error")
            return "break"
        try:
            self.client.chat(text, targets)
            self.composer.delete("1.0", "end")
        except (OSError, AttributeError, ProtocolError) as exc:
            self.append(f"发送失败：{exc}", "error")
        return "break"

    def start_server(self):
        if self.server:
            return
        try:
            port = int(self.server_port.get())
            if not 1 <= port <= 65535:
                raise ValueError("端口须在 1 至 65535 之间")
            self.server = NativeServer(host=self.server_host.get().strip(), port=port,
                    interface=self.server_interface.get().strip(), db=self.data / "accounts.json",
                    log=lambda text: self.events.put((None, "server_log", dict(message=text))),
                    on_exit=lambda: self.events.put((None, "server_stopped", {})))
            self.server.start()
            self.start_button.configure(state="disabled")
            self.stop_button.configure(state="normal")
        except (OSError, ValueError) as exc:
            self.server = None
            messagebox.showerror("服务器启动失败", str(exc), parent=self.root)

    def stop_server(self):
        server = self.server
        if server:
            threading.Thread(target=server.stop).start()
        self.stop_button.configure(state="disabled")

    def _poll(self):
        for _ in range(150):
            try:
                generation, event, body = self.events.get_nowait()
            except queue.Empty:
                break
            if generation is not None and generation != self.generation:
                continue
            if event == "server_stopped":
                if self.server and self.server.process.poll() is not None:
                    self.server = None
                    self.start_button.configure(state="normal")
                    self.stop_button.configure(state="disabled")
            elif event == "server_log":
                self.server_log.configure(state="normal")
                self.server_log.insert("end", datetime.now().strftime("%H:%M:%S  ") + body["message
```

```
> "] + "\n")
244                 self.server_log.see("end")
245                 self.server_log.configure(state="disabled")
246             elif event == "result":
247                 self.busy = False
248                 action = body.get("action")
249                 if action == "CHAT":
250                     continue
251                 self.append(body.get("message", ""))
252                 if body.get("ok") and action == "LOGIN":
253                     self._online(True)
254                     self.password.set("")
255                     self.status.set(f"已登录：{body['name']} · 群发使用 UDP 组播，未确认消息自动补
> 发")
256                 elif action == "LOGOUT":
257                     self._online(False)
258                     self.member_list.delete(0, "end")
259                     self.members = []
260                     self.member_label.configure(text="0 人在线")
261                     self.status.set("已注销 · 可以重新登录或切换账号")
262                 elif action == "REGISTER":
263                     self.status.set("注册成功，请点击登录" if body.get("ok") else "注册未完成，请检
> 查提示")
264             elif event == "roster":
265                 chosen = {self.members[i] for i in self.member_list.curselection()}
266                 self.members = body["members"]
267                 self.member_list.delete(0, "end")
268                 for i, name in enumerate(self.members):
269                     self.member_list.insert("end", name + ("（我）" if self.client and name == self
> .client.name else ""))
270                     if name in chosen:
271                         self.member_list.selection_set(i)
272                 self.member_label.configure(text=f"{len(self.members)} 人在线")
273             elif event == "message":
274                 when = datetime.fromtimestamp(body["ts"] / 1000).strftime("%H:%M:%S")
275                 scope = "群聊" if body["mode"] == "all" else "定向 → " + "、".join(body["targets"])
276                 self.append(f"{body['sender']}  {when}  [{scope} · {body['transport']}]", "sender"
> if body["mode"] == "all" else "private")
277                 self.append(body["text"] + "\n", "content")
278             elif event in {"error", "warning"}:
279                 self.busy = False
280                 self.append(body.get("message", "未知错误") + (f" [{body['code']}]" if "code" in bo
> dy else ""), "error")
281                 if not self.client or not self.client.name:
282                     self.status.set("操作未完成，请检查提示后重试")
283             elif event == "transport":
284                 self.append(body["message"])
285             elif event == "disconnected":
286                 self.busy = False
287                 self._online(False)
288                 self.members = []
289                 self.member_list.delete(0, "end")
290                 self.member_label.configure(text="0 人在线")
291                 self.status.set("连接已关闭 · 可以重新登录")
292         self.root.after(60, self._poll)
293 
294     def close(self):
295         if self.client:
296             self.client.close()
297         if self.server:
298             self.server.stop()
299         self.root.destroy()
300 
301 
302 def main():
303     parser = argparse.ArgumentParser(description="局域网聊天室图形界面")
```

```
    parser.add_argument("--data", default="data")
    args = parser.parse_args()
    root = tk.Tk()
    ChatApp(root, args.data)
    root.mainloop()


if __name__ == "__main__":
    main()
```

lan_chat/native_server.py

```
"""Launch and monitor the bundled C++ server without a console window."""
from pathlib import Path
import subprocess
import threading


class NativeServer:
    def __init__(self, host, port, db, interface, log=print, on_exit=lambda: None):
        self.host, self.port, self.db, self.interface = host, port, db, interface
        self.log, self.on_exit = log, on_exit
        self.process = None
        self.monitor = None

    def start(self):
        executable = Path(__file__).resolve().parent.parent / "bin" / "chat_server.exe"
        if not executable.exists():
            raise OSError("找不到 C++ 服务端，请先运行 build_server.bat")
        self.process = subprocess.Popen(
            [str(executable), "--host", self.host, "--port", str(self.port),
             "--interface", self.interface, "--db", str(self.db)],
            stdout=subprocess.PIPE, stderr=subprocess.STDOUT,
            text=True, encoding="utf-8", errors="replace",
            creationflags=getattr(subprocess, "CREATE_NO_WINDOW", 0))
        ready = threading.Event()
        failure = []
        process = self.process

        def monitor():
            for line in process.stdout:
                line = line.rstrip()
                self.log(line)
                if line.startswith("READY "):
                    ready.set()
                if line.startswith("FATAL "):
                    failure.append(line)
            process.wait()
            process.stdout.close()
            ready.set()
            self.on_exit()
        self.monitor = threading.Thread(target=monitor, daemon=True)
        self.monitor.start()
        if not ready.wait(5) or failure or self.process.poll() is not None:
            self.stop()
            raise OSError(failure[0] if failure else "C++ 服务端未能启动，请检查端口和网卡地址")

    def stop(self):
        if self.process and self.process.poll() is None:
            self.process.terminate()
            try:
                self.process.wait(timeout=3)
            except subprocess.TimeoutExpired:
                self.process.kill()
                self.process.wait(timeout=3)
        if self.monitor and self.monitor is not threading.current_thread():
            self.monitor.join(timeout=2)

```

启动与构建文件

lan_chat/__init__.py

```
"""LAN classroom chat. Standard library only."""

__version__ = "1.0.0"
```

lan_chat/__main__.py

```
from .gui import main

main()
```

start.bat

```
@echo off
chcp 65001 >nul
cd /d "%~dp0"
python -m lan_chat
if errorlevel 1 pause
```

start_server.bat

```
@echo off
chcp 65001 >nul
cd /d "%~dp0"
bin\chat_server.exe %*
if errorlevel 1 pause
```

build_server.bat

```
@echo off
chcp 65001 >nul
cd /d "%~dp0"
if not exist bin mkdir bin
where g++ >nul 2>nul
if errorlevel 1 (
  echo 请安装 MinGW-w64 并将 g++ 添加到 PATH，或使用 Visual Studio 开发者命令行：
  echo cl /std:c++17 /EHsc /utf-8 server_cpp\main.cpp /Fe:bin\chat_server.exe ws2_32.lib bcrypt.lib
  exit /b 1
)
g++ -std=c++17 -municode -O2 -Wall -Wextra -finput-charset=UTF-8 -fexec-charset=UTF-8 -static -stat
> ic-libgcc -static-libstdc++ server_cpp\main.cpp -o bin\chat_server.exe -lws2_32 -lbcrypt
if errorlevel 1 exit /b 1
echo C++ 服务端编译完成：bin\chat_server.exe
```

附录 B 自动测试完整源码

tests/test_chat.py

```python
"""Real C++ process + TCP/UDP integration tests, no simulated server."""
from collections import deque
import hashlib
import json
from pathlib import Path
import socket
import struct
import tempfile
import threading
import time
import unittest

from lan_chat.client import ChatClient
from lan_chat.native_server import NativeServer
from lan_chat.protocol import (MAX_FRAME, ProtocolError, canonical, decode, encode,
                               make_packet, recv_frame, send_frame)


class Inbox:
    def __init__(self):
        self.events = deque()
        self.condition = threading.Condition()

    def put(self, event, body):
        with self.condition:
            self.events.append((event, body))
            self.condition.notify_all()

    def wait(self, event, check=lambda body: True, timeout=5):
        deadline = time.monotonic() + timeout
        with self.condition:
            while True:
                for item in list(self.events):
                    if item[0] == event and check(item[1]):
                        self.events.remove(item)
                        return item[1]
                remaining = deadline - time.monotonic()
                if remaining <= 0:
                    raise AssertionError(f"Timed out waiting for {event}: {list(self.events)}")
                self.condition.wait(remaining)

    def has(self, event, check=lambda body: True):
        with self.condition:
            return any(e == event and check(b) for e, b in self.events)


class ProtocolTests(unittest.TestCase):
    def test_01_chinese_utf8_roundtrip(self):
        packet = make_packet("CHAT", dict(text="你好，局域网！Hello 🌏\n第二行"))
        self.assertEqual(decode(encode(packet)), packet)

    def test_02_tampering_detected(self):
        packet = make_packet("CHAT", dict(text="原文"))
        packet["body"]["text"] = "篡改"
        with self.assertRaises(ProtocolError) as ctx:
            decode(canonical(packet))
        self.assertEqual(ctx.exception.code, "BAD_CHECKSUM")

    def test_03_malformed_envelopes(self):
```

```
        for raw in (b"[]", b"{", b"\xff", b'{"v":1,"v":1}', b'{"x":NaN}'):
            with self.subTest(raw=raw), self.assertRaises(ProtocolError):
                decode(raw)
        with self.assertRaises(ProtocolError):
            decode(b" " * (MAX_FRAME + 1))


class NativeIntegrationTests(unittest.TestCase):
    @classmethod
    def setUpClass(cls):
        cls.temp = tempfile.TemporaryDirectory(prefix="lan_chat_test_")
        cls.db = Path(cls.temp.name) / "中文账户" / "accounts.json"
        with socket.socket() as probe:
            probe.bind(("127.0.0.1", 0))
            cls.port = probe.getsockname()[1]
        cls.logs = []
        cls.server = NativeServer("127.0.0.1", cls.port, cls.db, "127.0.0.1", cls.logs.append)
        cls.server.start()

    @classmethod
    def tearDownClass(cls):
        cls.server.stop()
        cls.temp.cleanup()

    def setUp(self):
        self.clients = []
        self.raw = []

    def tearDown(self):
        for client in self.clients:
            client.close()
        for sock in self.raw:
            sock.close()

    def account(self, name, multicast=True):
        inbox = Inbox()
        client = ChatClient(inbox.put, interface="127.0.0.1", multicast=multicast)
        self.clients.append(client)
        client.connect("127.0.0.1", self.port)
        client.register(name, "demo123456")
        result = inbox.wait("result", lambda b: b["action"] == "REGISTER")
        self.assertTrue(result["ok"], result)
        client.login(name, "demo123456")
        inbox.wait("result", lambda b: b["action"] == "LOGIN")
        return client, inbox

    def socket(self):
        sock = socket.create_connection(("127.0.0.1", self.port), timeout=3)
        self.raw.append(sock)
        return sock

    def response(self, sock, task, body=None):
        send_frame(sock, make_packet(task, body))
        return recv_frame(sock)

    def test_04_register_login_password_and_persistence(self):
        client, box = self.account("账户校验")
        stored = json.loads(self.db.read_text(encoding="utf-8"))["账户校验"]
        self.assertNotIn("demo123456", self.db.read_text(encoding="utf-8"))
        expected = hashlib.pbkdf2_hmac("sha256", b"demo123456", bytes.fromhex(stored["salt"]), 2000
 > 00).hex()
        self.assertEqual(stored["digest"], expected)
        sock = self.socket()
        self.assertFalse(self.response(sock, "REGISTER", dict(name="账户校验", password="demo123456
 > "))["body"]["ok"])
        self.assertEqual(self.response(sock, "LOGIN", dict(name="账户校验", password="wrong123"))["
```

```
 >  body"]["code"], "AUTH_FAILED")
124         self.assertEqual(self.response(sock, "LOGIN", dict(name="账户校验", password="demo123456"))
 >  ["body"]["code"], "ALREADY_ONLINE")
125         client.logout()
126         box.wait("result", lambda b: b["action"] == "LOGOUT")
127         self.assertTrue(self.response(sock, "LOGIN", dict(name="账户校验", password="demo123456"))[
 >  "body"]["ok"])
128 
129     def test_05_real_multicast_chinese_and_dedup(self):
130         a, ia = self.account("组播甲")
131         b, ib = self.account("组播乙")
132         text = "中文群聊：你好，世界！English 123 🙂"
133         a.chat(text)
134         ma = ia.wait("message", lambda m: m["text"] == text)
135         mb = ib.wait("message", lambda m: m["text"] == text)
136         self.assertEqual(ma["transport"], "UDP 组播")
137         self.assertEqual(mb["transport"], "UDP 组播")
138         self.assertEqual(ma["id"], mb["id"])
139         # A repeated server packet must not display twice.
140         repeated = make_packet("MESSAGE", {k: v for k, v in ma.items() if k not in {"id", "ts", "tr
 >  ansport"}})
141         repeated["id"] = ma["id"]
142         a._message(repeated, "TCP")
143         self.assertFalse(ia.has("message", lambda m: m["text"] == text))
144 
145     def test_06_selected_multiple_recipient_isolation(self):
146         a, ia = self.account("定向甲")
147         b, ib = self.account("定向乙")
148         c, ic = self.account("定向丙")
149         d, id_ = self.account("旁观丁")
150         a.chat("仅乙丙收到", ["定向乙", "定向丙"])
151         for box in (ia, ib, ic):
152             message = box.wait("message", lambda m: m["text"] == "仅乙丙收到")
153             self.assertEqual(message["transport"], "TCP")
154             self.assertEqual(message["mode"], "selected")
155         with self.assertRaises(AssertionError):
156             id_.wait("message", lambda m: m["text"] == "仅乙丙收到", timeout=0.35)
157 
158     def test_07_multicast_loss_tcp_repair(self):
159         a, ia = self.account("补发甲")
160         b, ib = self.account("补发乙", multicast=False)
161         start = time.monotonic()
162         a.chat("关闭组播仍收到")
163         msg = ib.wait("message", lambda m: m["text"] == "关闭组播仍收到")
164         self.assertEqual(msg["transport"], "TCP")
165         self.assertGreaterEqual(time.monotonic() - start, 0.65)
166 
167     def test_08_invalid_task_and_corruption_recover(self):
168         sock = self.socket()
169         self.assertEqual(self.response(sock, "TASK_DOES_NOT_EXIST")["body"]["code"], "INVALID_TASK"
 >  )
170         bad = make_packet("PING")
171         bad["body"]["tampered"] = True
172         raw = canonical(bad)
173         sock.sendall(struct.pack("!I", len(raw)) + raw)
174         self.assertEqual(recv_frame(sock)["body"]["code"], "BAD_CHECKSUM")
175         self.assertEqual(self.response(sock, "PING")["type"], "PONG")
176 
177     def test_09_fragmented_and_coalesced_tcp_frames(self):
178         sock = self.socket()
179         one, two = encode(make_packet("PING")), encode(make_packet("PING"))
180         frame = struct.pack("!I", len(one)) + one + struct.pack("!I", len(two)) + two
181         for index in range(0, len(frame), 7):
182             sock.sendall(frame[index:index+7])
183         self.assertEqual(recv_frame(sock)["type"], "PONG")
184         self.assertEqual(recv_frame(sock)["type"], "PONG")
```

```
185
186     def test_10_malformed_packet_and_oversized_frame(self):
187         sock = self.socket()
188         for raw in (b"not JSON", b"\xff", b'{"v":1,"v":2}'):
189             sock.sendall(struct.pack("!I", len(raw)) + raw)
190             self.assertEqual(recv_frame(sock)["body"]["code"], "BAD_PACKET")
191         self.assertEqual(self.response(sock, "CHAT", dict(text="匿名", mode="all"))["body"]["code"]
  > , "NOT_LOGGED_IN")
192         sock.sendall(struct.pack("!I", MAX_FRAME + 1))
193         self.assertEqual(recv_frame(sock)["body"]["code"], "FRAME_TOO_LARGE")
194         self.assertEqual(sock.recv(1), b"")
195         fresh = self.socket()
196         self.assertEqual(self.response(fresh, "PING")["type"], "PONG")
197
198     def test_11_bad_body_replay_and_roster_on_disconnect(self):
199         a, ia = self.account("边界甲")
200         b, ib = self.account("边界乙")
201         ia.wait("roster", lambda m: "边界乙" in m["members"])
202         for text, targets, code in [("", None, "BAD_BODY"), ("字" * 2001, None, "BAD_BODY"),
203                                     ("你好", [], "NO_RECIPIENT"), ("你好", ["不存在"], "NO_RECIPIEN
  > T")]:
204             a.chat(text, targets)
205             ia.wait("error", lambda m: m["code"] == code)
206         a.request("CHAT", dict(text="类型错误", mode="selected", targets=[123]))
207         ia.wait("error", lambda m: m["code"] == "BAD_BODY")
208         b.close()
209         ia.wait("roster", lambda m: "边界乙" not in m["members"])
210         raw = self.socket()
211         packet = make_packet("PING")
212         send_frame(raw, packet)
213         self.assertEqual(recv_frame(raw)["type"], "PONG")
214         send_frame(raw, packet)
215         self.assertEqual(recv_frame(raw)["body"]["code"], "REPLAY")
216
217     def test_12_database_survives_restart(self):
218         # Separate real process proves persistence independent of server memory.
219         temp = tempfile.TemporaryDirectory(prefix="lan_persist_")
220         with socket.socket() as probe:
221             probe.bind(("127.0.0.1", 0))
222             port = probe.getsockname()[1]
223         server = NativeServer("127.0.0.1", port, Path(temp.name) / "accounts.json", "127.0.0.1", la
  > mbda x: None)
224         try:
225             server.start()
226             with socket.create_connection(("127.0.0.1", port), timeout=3) as s:
227                 self.assertTrue(self.response(s, "REGISTER", dict(name="持久化", password="persist1
  > 23"))["body"]["ok"])
228             server.stop()
229             server.start()
230             with socket.create_connection(("127.0.0.1", port), timeout=3) as s:
231                 self.assertTrue(self.response(s, "LOGIN", dict(name="持久化", password="persist123"
  > ))["body"]["ok"])
232         finally:
233             server.stop()
234             temp.cleanup()
235
236
237 if __name__ == "__main__":
238     unittest.main(verbosity=2)
```